

LAPORAN
PROGRAM SISTEM PEMBELIAN TIKET BIOSKOP
PRODI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER

Disusun untuk Memenuhi UAS Mata Kuliah Algoritma dan Struktur Data

Dosen Pengampu:
Taufik Ridwan, S.T., M.T.



Disusun Oleh:

Cep Wildan Yanfa	2510631250057
Sarah Putri Aqilah	2510631250045
Imelvi Karnia Zalukhu	2510631250018
Dillan Gustav Alfaro	2510631250030

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan teknologi informasi saat ini telah memberikan banyak kemudahan dalam berbagai bidang, termasuk dalam pengelolaan data dan pelayanan di dunia hiburan seperti bioskop. Sistem pemesanan tiket bioskop yang sebelumnya dilakukan secara manual kini mulai beralih menjadi sistem digital agar proses pengelolaan data menjadi lebih cepat, efisien, dan terstruktur.

Dalam proses pengelolaan tiket bioskop, sering kali ditemukan kendala seperti pencatatan data film yang kurang rapi, kesulitan dalam mencari data, serta proses pengelolaan status tiket yang memakan waktu apabila masih dilakukan secara manual. Oleh karena itu, diperlukan sebuah sistem yang mampu membantu proses pengelolaan data film dan pemesanan tiket secara lebih efektif.

Berdasarkan permasalahan tersebut, dibuatlah sebuah program Sistem Pemesanan Tiket Bioskop berbasis Java. Program ini dirancang dengan menerapkan konsep *Object Oriented Programming* (OOP), struktur data *Array of Object*, serta algoritma *searching* dan *sorting* untuk mempermudah pengelolaan data film. Selain itu, program juga dilengkapi fitur penyimpanan data ke dalam file sehingga data dapat disimpan dan digunakan kembali.

Dengan adanya program ini, diharapkan proses pengelolaan data bioskop dapat dilakukan dengan lebih mudah, cepat, dan terorganisir.

1.2 TUJUAN

Adapun tujuan dari pembuatan program Sistem Pemesanan Tiket Bioskop ini adalah sebagai berikut:

1. Membuat sistem pengelolaan data film dan tiket bioskop berbasis Java.
2. Menerapkan konsep *Object Oriented Programming* (OOP) dalam pembuatan program.
3. Mengimplementasikan struktur data *Array of Object* dalam pengelolaan data.
4. Mengimplementasikan algoritma *searching* dan *sorting* untuk mempermudah pencarian dan pengurutan data.
5. Membuat sistem penyimpanan data menggunakan *file* agar data dapat disimpan secara permanen.
6. Mempermudah proses pengelolaan data film dan pemesanan tiket bioskop secara lebih efisien dan terstruktur.

BAB II

ANALISIS KEBUTUHAN

2.1 FITUR

Program Sistem Pemesanan Tiket Bioskop memiliki beberapa fitur utama yang digunakan untuk membantu pengelolaan data film dan tiket bioskop. Adapun fitur-fitur yang terdapat pada program adalah sebagai berikut:

1. Tambah Data Film

Fitur ini digunakan untuk menambahkan data film baru ke dalam sistem. Data yang dimasukkan meliputi ID film, nama film, kategori, status film, jumlah tiket terjual, dan jumlah tiket tersedia.

2. Tampilkan Data Film

Fitur ini digunakan untuk menampilkan seluruh data film yang tersimpan di dalam sistem.

3. Edit Data Film

Fitur ini digunakan untuk mengubah data film berdasarkan ID tertentu.

4. Hapus Data Film

Fitur ini digunakan untuk menghapus data film dengan metode soft delete menggunakan atribut aktif.

5. Pencarian Data

Program menyediakan beberapa fitur pencarian data, yaitu:

- a. Pencarian berdasarkan nama film
- b. Pencarian berdasarkan ID film
- c. Pencarian berdasarkan kategori film

6. Pengurutan Data

Program menyediakan fitur pengurutan data menggunakan algoritma sorting, yaitu:

- a. *Bubble Sort* berdasarkan ID
- b. *Selection Sort* berdasarkan nama film
- c. *Bubble Sort* berdasarkan tiket terjual

7. Statistik Film

Fitur ini digunakan untuk menampilkan total film, jumlah tiket tersedia, dan film dengan tiket terjual terbanyak.

8. Film Akan Tayang

Fitur ini digunakan untuk menampilkan daftar film dengan status “Akan Tayang”.

9. Update Status Film

Fitur ini digunakan untuk memperbarui status film yang tersedia di dalam sistem.

10. Counter Film

Fitur ini digunakan untuk menghitung jumlah total film, jumlah film sedang tayang, dan jumlah film yang akan tayang.

11. Simpan dan Load Data

Program memiliki fitur penyimpanan data ke *file* teks serta membaca kembali data dari *file*.

Berdasarkan fitur-fitur yang telah dibuat, program ini mampu membantu proses pengelolaan data film dan tiket bioskop secara lebih terstruktur dan efisien. Dengan adanya fitur pencarian, pengurutan, serta penyimpanan data, pengguna dapat mengelola informasi film dengan lebih mudah dan cepat.

2.2 STRUKTUR DATA

Program Sistem Pemesanan Tiket Bioskop menggunakan struktur data *Array of Object* untuk menyimpan data film.

Class Film digunakan sebagai struktur utama penyimpanan data dengan atribut:

- a. id
- b. namaFilm
- c. kategori
- d. statusFilm
- e. tiketTerjual
- f. tiketTersedia
- g. Aktif

Data film disimpan ke dalam *array object* berikut: *static Film[] daftarFilm = new Film[100];*

Array tersebut mampu menyimpan maksimal 100 data film.

Program juga menggunakan:

- a. *Scanner* untuk menerima input pengguna
- b. *File Handling* untuk menyimpan dan membaca data dari *file* teks
- c. *Algoritma Bubble Sort* dan *Selection Sort* untuk pengurutan data
- d. *Sequential Search* dan *Binary Search* untuk pencarian data

BAB III IMPLEMENTASI

3.1 Inisialisasi Data Awal

Metode main() merupakan titik awal eksekusi program. Di dalam metode ini terdapat perulangan do while yang membuat program terus berjalan hingga pengguna memilih menu keluar).

```
        public static void main(String[] args) {

            inisialisasi30Data();

            int pilih;

            do {

                System.out.println("\n=====
                =====");

                System.out.println("        SISTEM PEMESANAN TIKET BIOSKOP");

                System.out.println("=====
                =====");

                System.out.println("1. Tambah Data Film");

                System.out.println("2. Tampilkan Semua Data");

                System.out.println("3. Edit Data Film");

                System.out.println("4. Hapus Data Film");

                System.out.println("5. Cari Berdasarkan Nama");
```

```
System.out.println("6. Cari Berdasarkan ID");

System.out.println("7. Cari Berdasarkan Kategori");

System.out.println("8. Urutkan Berdasarkan ID");

System.out.println("9. Urutkan Berdasarkan Nama");

System.out.println("10. Urutkan Berdasarkan Tiket Terjual");

System.out.println("11. Statistik Film");

System.out.println("12. Film Akan Tayang");

System.out.println("13. Update Status Film");

System.out.println("14. Counter Film");

System.out.println("15. Simpan Data ke File");

System.out.println("16. Load Data dari File");

System.out.println("0. Keluar");
```

```
System.out.println("=====
=====");
```

```
System.out.print("Pilih Menu: ");
```

```
pilih = input.nextInt();
```

```
input.nextLine();
```

```
switch (pilih) {
```

```
    case 1: tambahData(); break;
```

```
        case 2: tampilData(); break;

        case 3: editData(); break;

        case 4: hapusData(); break;

        case 5: cariNama(); break;

        case 6: cariID(); break;

        case 7: cariKategori(); break;

        case 8: bubbleSortID(); tampilData(); break;

        case 9: selectionSortNama(); tampilData(); break;

        case 10: bubbleSortTiket(); tampilData(); break;

        case 11: statistikFilm(); break;

        case 12: filmAkanTayang(); break;

        case 13: updateStatus(); break;

        case 14: counterFilm(); break;

        case 15: simpanKeFile(); break;

        case 16: loadDariFile(); break;

        case 0: System.out.println("Keluar..."); break;

        default: System.out.println("Pilihan salah!");

    }

    } while (pilih != 0);

}
```

metode inisialisasi30Data() dipanggil untuk memuat 30 data film awal. Kemudian program menampilkan menu dalam bentuk teks dan menunggu input dari pengguna. Input berupa angka menu yang dipilih akan diproses menggunakan statement switch case.

3.2 Fungsi Fungsi Utama Sistem

3.3.1 Tambah Data Film (‘tambah Data () ’)

Metode ini menerima input dari pengguna untuk membuat data film baru dan menyimpannya ke dalam array daftarFilm.

```
static void tambahData() {  
  
    System.out.print("ID: ");  
  
    int id = input.nextInt();  
  
    input.nextLine();  
  
  
    System.out.print("Nama: ");  
  
    String nama = input.nextLine();  
  
    System.out.print("Kategori: ");  
  
    String kat = input.nextLine();  
  
    System.out.print("Status: ");  
  
    String stat = input.nextLine();  
  
    System.out.print("Terjual: ");
```

```

int terjual = input.nextInt();

System.out.print("Tersedia: ");

int sedia = input.nextInt();

daftarFilm[jumlahData++] = new Film(id, nama, kat, stat, terjual, sedia);

System.out.println("Data ditambahkan.");

}

```

Input diterima secara bertahap: Id, nama, kategori, status, tiket terjual, dan tiket tersedia. Lalu data ini dimasukkan ke array dan 'jumlahData' bertambah.

3.3.2 Menampilkan Semua Data Film('**tampil Data()**')

```

static void tampilData() {

    System.out.printf("%-4s    %-25s    %-10s    %-15s    %-10s\n", "ID", "NAMA", "KATEGORI", "STATUS", "JUAL", "SISA");

    for (int i = 0; i < jumlahData; i++) {

        if (daftarFilm[i].aktif) {

            System.out.printf("%-4d %-25s %-10s %-15s %-10d %-10d\n",

                daftarFilm[i].id, daftarFilm[i].namaFilm, daftarFilm[i].kategori,

                daftarFilm[i].statusFilm, daftarFilm[i].tiketTerjual,

                daftarFilm[i].tiketTersedia);

```

```

    }

}

}

```

Menampilkan data film yang aktif dalam bentuk tabel (ID, Nama film, Kategori, Status, Tiket terjual, dan sisa tiket).

3.3.3 Edit Data Film (‘ edit Data() ’)

```

static void editData() {

    System.out.print("ID edit: ");

    int cari = input.nextInt();

    input.nextLine();

    for (int i = 0; i < jumlahData; i++) {

        if (daftarFilm[i].id == cari &&daftarFilm[i].aktif) {

            System.out.print("Nama baru: ");

            daftarFilm[i].namaFilm = input.nextLine();

            System.out.print("Terjual baru: ");

            daftarFilm[i].tiketTerjual = input.nextInt();

            System.out.println("Update sukses.");

            return;

```

```

    }

}

System.out.println("Tidak ditemukan.");

}

```

Melakukan pencarian film berdasarkan ID. Jika ditemukan dan status aktif, nama dan tiket terjual bisa diubah manual.

3.3.4 Hapus Data Film(' hapus Data() ')

```

static void hapusData() {

    System.out.print("ID hapus: ");

    int cari = input.nextInt();

    for (int i = 0; i < jumlahData; i++) {

        if (daftarFilm[i].id == cari) {

            daftarFilm[i].aktif = false;

            System.out.println("Dihapus.");

            return;

        }

    }

}

```

Hapus data dilakukan dengan menandai aktif = false. Data tetap ada di array tapi tidak ditampilkan.

3.3.5 Cari Film Berdasarkan Nama ('cari Nama()')

```
static void cariNama( ) {  
  
    System.out.print("Cari Nama: ");  
  
    String cari = input.nextLine();  
  
    for (int i = 0; i < jumlahData; i++) {  
  
                                                if      (daftarFilm[i].aktif      &&  
daftarFilm[i].namaFilm.toLowerCase().contains(cari.toLowerCase())) {  
  
        System.out.println(daftarFilm[i].namaFilm);  
  
        }  
  
    }  
  
}
```

Melakukan pencarian nama film yang mengandung kata kunci input .

3.3.6 Cari Film Berdasarkan Id (' cari ID () ')

```
static void cariID() {  
  
    bubbleSortID();  
  
    System.out.print("Cari ID: ");  
  
    int cari = input.nextInt();  
  
    int l = 0, r = jumlahData - 1;
```

```

while (l <= r) {

    int m = (l + r) / 2;

    if (daftarFilm[m].id == cari) {

        System.out.println(daftarFilm[m].namaFilm);

        return;

    } else if (daftarFilm[m].id < cari) {

        l = m + 1;

    } else {

        r = m - 1;

    }

}

System.out.println("Tidak ditemukan.");

}

```

Menggunakan algoritma binary search untuk pencarian data berdasarkan ID yang sudah diurutkan (bubble sort ID dipanggil sebelum pencarian).

3.3.7 Cari Film Berdasarkan Kategori (‘ cariKategori () ’)

```

static void cariKategori() {

    System.out.print("Cari Kategori: ");

    String cari = input.nextLine();

    for (int i = 0; i < jumlahData; i++) {

```

```

        if (daftarFilm[i].aktif && daftarFilm[i].kategori.equalsIgnoreCase(cari)) {
            System.out.println(daftarFilm[i].namaFilm);

        }

    }

}

```

Mencari film yang memiliki kategori sama dengan input user .

3.3.8 Urutkan Data Berdasarkan ID

```

static void bubbleSortID( ) {

    for (int i = 0; i < jumlahData -1; i++) {

        for (int j = 0; j < jumlahData - i -1; j++) {

            if (daftarFilm[j].id > daftarFilm[j+1].id){

                Film t = daftarFilm[j];

                daftarFilm[j] = daftarFilm[j+1];

                daftarFilm[j+1] = t;

            }

        }

    }

}

```

Bubble sort mengurutkan film berdasarkan ID secara ascending.

3.3.9 Urutkan Data Berdasarkan Nama

```
static void selectionSortNama( ) {  
  
    for (int i = 0; i < jumlahData - 1; i++) {  
  
        int min = i;  
  
        for (int j = i + 1; j < jumlahData; j++) {  
  
                                                    if  
(daftarFilm[j].namaFilm.compareToIgnoreCase(daftarFilm[min].namaFilm) < 0)  
{  
  
                min = j;  
  
            }  
  
        }  
  
        Film t = daftarFilm[min];  
  
        daftarFilm[min] = daftarFilm[i];  
  
        daftarFilm[i] = t;  
  
    }  
  
}
```

Selection sort mengurutkan film berdasarkan nama secara alfabetis.

3.3.10 Urutkan Data Berdasarkan Tiket Terjual

```
static void bubbleSortTiket( ) {
```

```

for (int i = 0; i < jumlahData - 1; i++) {

    for (int j = 0; j < jumlahData - i - 1; j++) {

        if (daftarFilm[j].tiketTerjual < daftarFilm[j + 1].tiketTerjual) {

            Film t = daftarFilm[j];

            daftarFilm[j] = daftarFilm[j + 1];

            daftarFilm[j + 1] = t;

        }

    }

}

```

Bubble sort mengurutkan data tiket terjual dari yang terbanyak ke jumlah yang sedikit.

3.3.11 Statistik Film (‘ statistik Film () ’)

```

static void statistikFilm() {

    int total = 0, sisa = 0, max = -1, idx = -1;

    for(int i=0; i<jumlahData; i++) {

        if(daftarFilm[i].aktif) {

            total++;

            sisa += daftarFilm[i].tiketTersedia;

```

```

        if(daftarFilm[i].tiketTerjual > max) {

            max = daftarFilm[i].tiketTerjual;

            idx = i;

        }

    }

}

System.out.println("Total Film : " + total);

System.out.println("Tiket Tersedia : " + sisa);

if (idx != -1) {

    System.out.println("Terlaris : " + daftarFilm[idx].namaFilm);

}

}

```

Menampilkan statistik jumlah film aktif, total tiket tersedia, dan film dengan tiket terjual terbanyak.

3.3.12 Menampilkan Film Akan Tayang (‘ filmAkanTayang() ’)

```

static void filmAkanTayang() {

    System.out.println("\nFILM AKAN TAYANG:");

    for(int i=0; i<jumlahData; i++) {

        if(daftarFilm[i].statusFilm.equalsIgnoreCase("Akan Tayang")) {

            System.out.println(daftarFilm[i].namaFilm);

        }

    }

}

```

```

    }

}

```

Mencetak daftar film yang berstatus "Akan Tayang".

3.3.13 Update Status Film ('updateStatus ()')

```

static void updateStatus() {

    System.out.print("ID: ");

    int id = input.nextInt();

    input.nextLine();

    for(int i=0; i<jumlahData; i++) {

        if(daftarFilm[i].id == id) {

            System.out.print("Status baru: ");

            daftarFilm[i].statusFilm = input.nextLine();

            System.out.println("Status berhasil diupdate.");

            return;

        }

    }

}

```

Memperbaharui status tayang film berdasarkan input ID.

3.3.14 Counter Film Berdasarkan Status (' counterFilm ()')

```
static void counterFilm() {  
  
    int total=0, sedang=0, akan=0;  
  
    for(int i=0; i<jumlahData; i++) {  
  
        if(daftarFilm[i].aktif) {  
  
            total++;  
  
            if(daftarFilm[i].statusFilm.equalsIgnoreCase("Sedang Tayang")) {  
  
                sedang++;  
  
            }  
  
            if(daftarFilm[i].statusFilm.equalsIgnoreCase("Akan Tayang")) {  
  
                akan++;  
  
            }  
  
        }  
  
    }  
  
    System.out.println("Total Film : " + total);  
  
    System.out.println("Sedang Tayang : " + sedang);  
  
    System.out.println("Akan Tayang : " + akan);  
  
}
```

Menghitung total film aktif dan jumlah film berdasar status "Sedang Tayang" dan "Akan Tayang".

3.3.15 Simpan Data Ke File('simpanKeFile() ')

```
static void simpanKeFile() {  
  
    try {  
  
        PrintWriter w = new PrintWriter("datafilm.txt");  
  
        for(int i=0; i<jumlahData; i++) {  
  
            if(daftarFilm[i].aktif) {  
  
                w.println(daftarFilm[i].id + ";" +  
  
                    daftarFilm[i].namaFilm + ";" +  
  
                    daftarFilm[i].kategori + ";" +  
  
                    daftarFilm[i].statusFilm + ";" +  
  
                    daftarFilm[i].tiketTerjual + ";" +  
  
                    daftarFilm[i].tiketTersedia);  
  
            }  
  
        }  
  
        w.close();  
  
        System.out.println("Tersimpan.");  
  
    } catch(Exception e) {  
  
        System.out.println("Gagal save.");  
  
    }  
  
}
```

Menyimpan seluruh data film aktif ke dalam file dengan format “;”.

3.3.16 Load Data Dari File ('loadDariFile()')

```
static void loadDariFile() {  
  
    try{  
  
        Scanner b = new Scanner(new File("datafilm.txt"));  
  
        jumlahData = 0;  
  
        while(b.hasNextLine()) {  
  
            String[] d = b.nextLine().split(";");  
  
            daftarFilm[jumlahData++] = new Film(  
  
                Integer.parseInt(d[0]), d[1], d[2],d[3],  
  
                Integer.parseInt(d[4]),  Integer.parseInt(d[5]) );  
  
        }  
  
        b.close();  
  
        System.out.println("Loaded.");  
  
    } catch(Exception e) {  
  
        System.out.println("File tidak ada.");  
  
    }  
  
}
```

Membaca file teks datafilm.txt dan menginput data ke dalam array daftarFilm untuk digunakan dalam aplikasi.

BAB IV

ANALISIS KOMPLEKSITAS

Nama Metode/Fitur	Kompleksitas Waktu (Worst Case)	Kompleksitas Ruang (space)	Penjelasan
tambahData()	$O(1)$	$O(1)$	Memasukkan data langsung ke indeks terakhir (jumlahData++).
tampilData()	$O(n)$	$O(1)$	Melakukan satu kali perulangan (for) dari indeks 0 hingga n-1.
editData()	$O(n)$	$O(1)$	Kasus terburuk harus mencari data hingga indeks terakhir.
hapusData()	$O(n)$	$O(1)$	Menggunakan <i>Soft Delete</i> (mengubah flag aktif) pencarian ID membutuhkan satu perulangan.
cariNama()	$O(n)$	$O(1)$	Melakukan perulangan penuh untuk mengecek kesesuaian teks (<i>substring matching</i>).
cariKategori()	$O(n)$	$O(1)$	Melakukan perulangan penuh untuk membandingkan kategori.
cariID() (Binary Search)	$O(n^2)$	$O(1)$	penting: karena metode ini memanggil

			<i>bubbleSortID()</i> diawal, kompleksitas totalnya ditentukan oleh proses pengurutan ($O(n^2)$), bukan pencariannya ($O(\log n)$).
bubleSortID()	$O(n^2)$	$O(1)$	Menggunakan <i>nested loop</i> (perulangan bersarang) berukuran $n \times n$.
selectionSortNama()	$O(n^2)$	$O(1)$	Menggunakan <i>nested loop</i> untuk mencari nilai minimum dan menukarnya.
bubleSortTiket()	$O(n^2)$	$O(1)$	Menggunakan <i>nested loop</i> berukuran $n \times n$.
statistikFilm()	$O(n)$	$O(1)$	Melakukan satu kali perulangan untuk menghitung total, sisa, dan mencari nilai maksimum.
filmAkanTayang()	$O(n)$	$O(1)$	Melakukan satu kali perulangan untuk menyaring status film.
updateStatus()	$O(n)$	$O(1)$	Kasus terburuk harus mencari ID hingga akhir <i>array</i> .
counterFilm()	$O(n)$	$O(1)$	Melakukan perulangan penuh untuk menghitung kategori status.
simpanKeFile()	$O(n)$	$O(1)$	Melakukan

			perulangan sebanyak n kali untuk menulis baris data ke file teks.
loadDariFile()	$O(n)$	$O(1)$	Membaca baris demi baris dari file eksternal sebanyak jumlah baris yang ada (n).
inisialisasi30Data ()	$O(1)$	$O(1)$	Operasi konstan karena jumlah data yang dimasukkan sudah pasti 30 data.

BAB V

PENGUJIAN (OUTPUT BERBAGAI SKENARIO)

- Menampilkan Semua Data

```
=====
```

ID	NAMA	KATEGORI	STATUS	JUAL	SISA
1	Avengers Endgame	Action	Sedang Tayang	95	5
2	Spider Man No Way Home	Action	Sedang Tayang	88	12
3	The Batman	Action	Sedang Tayang	76	24
4	Fast Furious 9	Action	Akan Tayang	65	35
5	John Wick 4	Action	Sedang Tayang	90	10
6	Insidious	Horror	Sedang Tayang	70	30
7	The Nun	Horror	Akan Tayang	50	50
8	Annabelle	Horror	Sedang Tayang	45	55
9	Pengabdi Setan	Horror	Sedang Tayang	85	15
10	KKN Desa Penari	Horror	Akan Tayang	92	8
11	Frozen 2	Animasi	Sedang Tayang	60	40
12	Toy Story 4	Animasi	Sedang Tayang	58	42
13	Minions	Animasi	Akan Tayang	40	60
14	Moana	Animasi	Sedang Tayang	75	25
15	Encanto	Animasi	Akan Tayang	35	65
16	Interstellar	SciFi	Sedang Tayang	80	20
17	Inception	SciFi	Sedang Tayang	89	11
18	Oppenheimer	SciFi	Akan Tayang	55	45
19	Top Gun Maverick	Action	Sedang Tayang	93	7
20	Mission Impossible	Action	Sedang Tayang	82	18
21	Doctor Strange	Action	Akan Tayang	67	33
22	Black Panther	Action	Sedang Tayang	91	9
23	Aquaman	Action	Sedang Tayang	73	27
24	Transformers	SciFi	Akan Tayang	64	36
25	Joker	Drama	Sedang Tayang	78	22
26	Deadpool	Comedy	Sedang Tayang	87	13
27	Cars	Animasi	Sedang Tayang	49	51
28	Up	Animasi	Akan Tayang	52	48
29	Coco	Animasi	Sedang Tayang	69	31
30	Elemental	Animasi	Akan Tayang	44	56

- Searching Berdasarkan Nama

```
=====
Pilih Menu: 6
Cari ID: 25
Joker
```

- Sorting Berdasarkan Nama

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

8. Urutkan Berdasarkan ID
9. Urutkan Berdasarkan Nama
10. Urutkan Berdasarkan Tiket Terjual
11. Statistik Film
12. Film Akan Tayang
13. Update Status Film
14. Counter Film
15. Simpan Data ke File
16. Load Data dari File
0. Keluar

=====
Pilih Menu: 6
Cari ID: 25
Joker
```

- Statistik Film

```
=====
                        SISTEM PEMESANAN TIKET BIOSKOP
=====
1. Tambah Data Film
2. Tampilkan Semua Data
3. Edit Data Film
4. Hapus Data Film
5. Cari Berdasarkan Nama
6. Cari Berdasarkan ID
7. Cari Berdasarkan Kategori
8. Urutkan Berdasarkan ID
9. Urutkan Berdasarkan Nama
10. Urutkan Berdasarkan Tiket Terjual
11. Statistik Film
12. Film Akan Tayang
13. Update Status Film
14. Counter Film
15. Simpan Data ke File
16. Load Data dari File
0. Keluar
=====
Pilih Menu: 11
Total Film : 30
Tiket Tersedia : 903
Terlaris : Avengers Endgame
```

BAB VI

KESIMPULAN

Program Sistem Pemesanan Tiket Bioskop berbasis Java ini berhasil dibuat dengan menerapkan konsep Object Oriented Programming (OOP), struktur data Array of Object, serta algoritma searching dan sorting. Program ini memiliki berbagai fitur seperti tambah data, tampil data, edit data, hapus data, pencarian data, pengurutan data, statistik film, serta penyimpanan data ke file teks. Dengan adanya fitur-fitur tersebut, pengelolaan data film dan tiket bioskop menjadi lebih mudah dan terstruktur.

Dalam proses pengembangan program, digunakan beberapa algoritma seperti Bubble Sort, Selection Sort, Sequential Search, dan Binary Search untuk membantu proses pencarian dan pengurutan data. Selain itu, program juga menggunakan file handling untuk menyimpan dan memuat data sehingga data dapat digunakan kembali ketika program dijalankan. Penerapan algoritma dan struktur data pada program ini membantu meningkatkan efisiensi dalam pengolahan data film.

Berdasarkan hasil pengujian yang telah dilakukan, seluruh fitur dalam program dapat berjalan dengan baik sesuai dengan tujuan yang diharapkan. Program ini diharapkan dapat menjadi media pembelajaran dalam memahami penerapan algoritma dan struktur data menggunakan bahasa pemrograman Java, khususnya dalam pengelolaan data pada sistem pemesanan tiket bioskop.